

第 10 周：变分法与最优控制

从优化函数到优化路径——连续时间优化的统一视角

目录

1 泛函优化：从优化点到优化函数	3
2 Euler-Lagrange 方程：泛函的极值条件	5
3 最大熵原理：约束下的分布选择	14
4 KL 散度与变分推断	17
5 信息瓶颈	20
6 最优控制与 HJB 方程	22
7 HJB $\rightarrow$ Bellman, 伴随 $\rightarrow$ 反向传播	24
8 实验	27
9 本周总结	31
10 思考题	32

本讲导读

本周的核心洞察

之前的优化：

- 优化向量 $x \in \mathbb{R}^n$
- $\min_x f(x)$
- 梯度下降、牛顿法、SGD

本周的优化：

- 优化函数 $y(x)$ 或路径 $x(t)$
- $\min_{y(\cdot)} J[y] = \int F(y, y', x) dx$
- Euler-Lagrange 方程、HJB 方程

惊人联系：

- 最大熵 $\Rightarrow$ 指数族分布
- 变分推断 $\Rightarrow$ ELBO
- HJB 方程 $\Rightarrow$ Bellman 方程 (RL 的核心!)
- 伴随方法 $\Rightarrow$ 反向传播

内容结构

第一部分：泛函优化基础 (10min)

第二部分：Euler-Lagrange 方程 (15min)

第三部分：最大熵原理 (15min)

第四部分：KL 散度与变分推断 (15min)

第五部分：信息瓶颈 (15min)

第六部分：最优控制与 HJB (20min)

第七部分：HJB $\rightarrow$ Bellman, 伴随 $\rightarrow$ 反向传播 (20min)

第八部分：实验 (30min)

1 泛函优化：从优化点到优化函数

什么是泛函？

回顾：

- **函数** $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ，输入向量，输出数
- 例： $f(x_1, x_2) = x_1^2 + x_2^2$

新概念：

- **泛函** $J: \mathcal{F} \rightarrow \mathbb{R}$ ，输入函数，输出数
- $\mathcal{F}$ 是某个函数空间（如所有连续函数）

例子：

$$J[y] = \int_a^b F(y(x), y'(x), x) dx$$

输入一个函数 $y(x)$ ，输出一个数（积分值）

泛函的具体例子

例 1：曲线长度

从 $(0, 0)$ 到 $(1, 1)$ 的曲线 $y(x)$ ，长度是多少？

$$L[y] = \int_0^1 \sqrt{1 + (y'(x))^2} dx$$

这里 $F(y, y', x) = \sqrt{1 + (y')^2}$

具体计算： 如果 $y(x) = x$ （直线）

$$y'(x) = 1, \quad L = \int_0^1 \sqrt{1 + 1} dx = \sqrt{2} \approx 1.414$$

如果 $y(x) = x^2$ （抛物线）

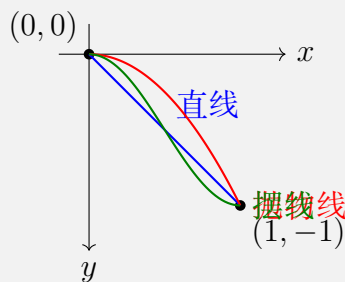
$$y'(x) = 2x, \quad L = \int_0^1 \sqrt{1 + 4x^2} dx \approx 1.479$$

直线更短！（符合直觉）

泛函的具体例子（续）

例 2：最速降线（Brachistochrone）

小球从 $(0, 0)$ 滑到 $(1, -1)$ ，哪条路径最快？



时间泛函： $T[y] = \int_0^1 \frac{\sqrt{1+(y')^2}}{\sqrt{-2gy}} dx$

答案：摆线 (Cycloid)! 不是直线也不是抛物线

泛函的具体例子 (续)

例 3：信息熵

给定概率分布 $p(x)$ ，熵是多少？

$$H[p] = - \int p(x) \log p(x) dx$$

输入一个概率密度函数 $p(x)$ ，输出一个数（熵值）

例 4：KL 散度

两个分布 p, q 的“距离”：

$$D_{\text{KL}}[p||q] = \int p(x) \log \frac{p(x)}{q(x)} dx$$

这些都是泛函！

泛函优化的目标

问题：在所有满足某些条件的函数中，找使泛函取极值的函数

普通优化：

$$\min_{x \in \mathbb{R}^n} f(x) \Rightarrow \nabla f(x^*) = 0$$

泛函优化：

$$\min_{y(\cdot) \in \mathcal{F}} J[y] \Rightarrow ??$$

需要回答：

“泛函的梯度”是什么？

“泛函的极值条件”是什么？

答案：Euler-Lagrange 方程！

2 Euler-Lagrange 方程：泛函的极值条件

变分的概念

普通微分： $x \rightarrow x + \epsilon$

$$df = f(x + \epsilon) - f(x) \approx f'(x) \cdot \epsilon$$

泛函的变分： $y(x) \rightarrow y(x) + \epsilon\eta(x)$

其中 $\eta(x)$ 是“扰动函数”， $\eta(a) = \eta(b) = 0$ （边界不动）

$$\delta J = J[y + \epsilon\eta] - J[y]$$

极值条件： 对所有扰动 η ， $\delta J = 0$

推导 Euler-Lagrange 方程

设泛函： $J[y] = \int_a^b F(y, y', x) dx$

扰动： $\tilde{y}(x) = y(x) + \epsilon\eta(x)$

$$J[y + \epsilon\eta] = \int_a^b F(y + \epsilon\eta, y' + \epsilon\eta', x) dx$$

Taylor 展开：

$$F(y + \epsilon\eta, y' + \epsilon\eta', x) \approx F + \epsilon\eta \frac{\partial F}{\partial y} + \epsilon\eta' \frac{\partial F}{\partial y'}$$

变分：

$$\delta J = \epsilon \int_a^b \left(\eta \frac{\partial F}{\partial y} + \eta' \frac{\partial F}{\partial y'} \right) dx$$

推导（续）：分部积分

对第二项分部积分：

$$\int_a^b \eta' \frac{\partial F}{\partial y'} dx = \left[\eta \frac{\partial F}{\partial y'} \right]_a^b - \int_a^b \eta \frac{d}{dx} \left(\frac{\partial F}{\partial y'} \right) dx$$

边界项 = 0（因为 $\eta(a) = \eta(b) = 0$ ）

代入：

$$\delta J = \epsilon \int_a^b \eta(x) \left(\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} \right) dx$$

极值条件： 对所有 η ， $\delta J = 0$

$\Rightarrow$ 括号内必须为零！

Euler-Lagrange 方程

核心结论：Euler-Lagrange 方程

$$\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} = 0$$

这是泛函 $J[y] = \int F(y, y', x) dx$ 取极值的必要条件
 类比：

普通优化	泛函优化
$\nabla f(x^*) = 0$	$\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} = 0$
代数方程组	微分方程

泛函优化变成了求解微分方程！

另一种推导：离散化 + 拉格朗日乘子

思路：把连续问题离散化，用普通优化方法，再取极限
 为什么这样做？

- 离散优化更直观（有限维）
- 用熟悉的拉格朗日乘子法
- 取极限得到连续结果
- 数值方法的理论基础

离散化设置

把区间 $[a, b]$ 分成 N 份：

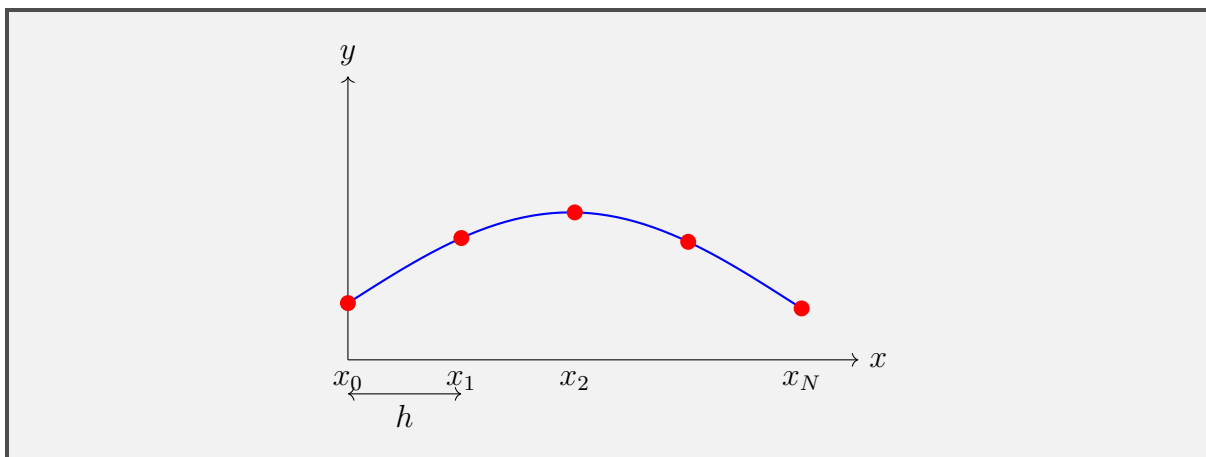
$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_N = b$$

其中 $h = \frac{b-a}{N}$ （步长）

离散化函数： $y(x) \rightarrow (y_0, y_1, y_2, \dots, y_N)$

离散化导数： $y'(x_i) \approx \frac{y_{i+1} - y_i}{h}$

离散化积分： $\int_a^b F dx \approx \sum_{i=0}^{N-1} F_i \cdot h$



离散化泛函

连续泛函： $J[y] = \int_a^b F(y, y', x) dx$

离散泛函：

$$J_N(y_0, y_1, \dots, y_N) = \sum_{i=0}^{N-1} F\left(y_i, \frac{y_{i+1} - y_i}{h}, x_i\right) \cdot h$$

边界条件： $y_0 = y(a)$, $y_N = y(b)$ 固定

优化变量： $y_1, y_2, \dots, y_{N-1}$ (内部点)

这是一个普通的 $(N-1)$ 维优化问题！

可以用 $\nabla J_N = 0$ 求极值

对内部点求导

对 y_k ($1 \leq k \leq N-1$) 求偏导：

y_k 出现在两项中：

- 第 $k-1$ 项： $F\left(y_{k-1}, \frac{y_k - y_{k-1}}{h}, x_{k-1}\right)$
- 第 k 项： $F\left(y_k, \frac{y_{k+1} - y_k}{h}, x_k\right)$

记： $F_i = F\left(y_i, \frac{y_{i+1} - y_i}{h}, x_i\right)$

求导：

$$\frac{\partial J_N}{\partial y_k} = \frac{\partial F_{k-1}}{\partial y_k} \cdot h + \frac{\partial F_k}{\partial y_k} \cdot h$$

展开偏导数

对于 F_{k-1} ： y_k 只出现在“导数”位置

$$\frac{\partial F_{k-1}}{\partial y_k} = \frac{\partial F}{\partial y'} \Big|_{k-1} \cdot \frac{\partial}{\partial y_k} \left(\frac{y_k - y_{k-1}}{h} \right) = \frac{\partial F}{\partial y'} \Big|_{k-1} \cdot \frac{1}{h}$$

对于 F_k ： y_k 出现在两个位置

$$\frac{\partial F_k}{\partial y_k} = \frac{\partial F}{\partial y} \Big|_k + \frac{\partial F}{\partial y'} \Big|_k \cdot \frac{\partial}{\partial y_k} \left(\frac{y_{k+1} - y_k}{h} \right) = \frac{\partial F}{\partial y} \Big|_k - \frac{\partial F}{\partial y'} \Big|_k \cdot \frac{1}{h}$$

极值条件

$$\frac{\partial J_N}{\partial y_k} = 0:$$

$$\frac{\partial F}{\partial y'} \Big|_{k-1} \cdot \frac{1}{h} \cdot h + \left(\frac{\partial F}{\partial y} \Big|_k - \frac{\partial F}{\partial y'} \Big|_k \cdot \frac{1}{h} \right) \cdot h = 0$$

简化：

$$\frac{\partial F}{\partial y'} \Big|_{k-1} + \frac{\partial F}{\partial y} \Big|_k \cdot h - \frac{\partial F}{\partial y'} \Big|_k = 0$$

整理：

$$\frac{\partial F}{\partial y} \Big|_k = \frac{1}{h} \left(\frac{\partial F}{\partial y'} \Big|_k - \frac{\partial F}{\partial y'} \Big|_{k-1} \right)$$

取连续极限

右边是差分：

$$\frac{1}{h} \left(\frac{\partial F}{\partial y'} \Big|_k - \frac{\partial F}{\partial y'} \Big|_{k-1} \right) = \frac{\frac{\partial F}{\partial y'}(x_k) - \frac{\partial F}{\partial y'}(x_{k-1})}{h}$$

当 $h \rightarrow 0$ 时，差商变导数：

$$\lim_{h \rightarrow 0} \frac{\frac{\partial F}{\partial y'}(x+h) - \frac{\partial F}{\partial y'}(x)}{h} = \frac{d}{dx} \frac{\partial F}{\partial y'}$$

因此：

$$\frac{\partial F}{\partial y} = \frac{d}{dx} \frac{\partial F}{\partial y'}$$

核心结论：离散推导得到 E-L 方程

$$\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} = 0$$

和变分法得到的完全一样！

数值例子：离散最短路径

问题：从 $(0,0)$ 到 $(1,1)$ 的最短路径

离散化： $N = 4, h = 0.25$

点： $(0, y_0), (0.25, y_1), (0.5, y_2), (0.75, y_3), (1, y_4)$

边界： $y_0 = 0, y_4 = 1$

离散长度：

$$L_4 = \sum_{i=0}^3 \sqrt{h^2 + (y_{i+1} - y_i)^2} = \sum_{i=0}^3 \sqrt{0.0625 + (y_{i+1} - y_i)^2}$$

对 y_1 求导：

$$\frac{\partial L_4}{\partial y_1} = \frac{y_1 - y_0}{\sqrt{0.0625 + (y_1 - y_0)^2}} - \frac{y_2 - y_1}{\sqrt{0.0625 + (y_2 - y_1)^2}}$$

数值例子（续）

极值条件： $\frac{\partial L_4}{\partial y_k} = 0$ 对 $k = 1, 2, 3$

令： $s_i = \frac{y_{i+1} - y_i}{\sqrt{0.0625 + (y_{i+1} - y_i)^2}}$ （归一化斜率）

条件变成： $s_0 = s_1 = s_2 = s_3$

即：所有线段斜率相同！

解： $y_1 = 0.25, y_2 = 0.5, y_3 = 0.75$

（等差数列 $\Rightarrow$ 直线！）

验证： $L_4 = 4 \times \sqrt{0.0625 + 0.0625} = 4 \times 0.354 = 1.414$

和 $\sqrt{2}$ 一致！

两种推导方法的比较

	变分法	离散化
起点	函数空间的扰动	有限维优化
工具	分部积分	偏导数链式法则
极限	无（直接得到）	$h \rightarrow 0$
直觉	数学上更优美	更接近数值计算
应用	理论分析	数值方法设计

两条路殊途同归：都得到 E-L 方程！

离散方法的额外好处：

- 直接给出数值算法（离散 E-L 方程组）
- 对于复杂问题，可能更容易推导
- 和现代优化方法（如自动微分）的联系更紧密

例 1：最短路径

问题：从 $(0, y_0)$ 到 $(1, y_1)$ ，哪条曲线最短？

$$L[y] = \int_0^1 \sqrt{1 + (y')^2} dx$$

这里 $F = \sqrt{1 + (y')^2}$

计算偏导：

$$\frac{\partial F}{\partial y} = 0 \quad (F \text{ 不显含 } y)$$

$$\frac{\partial F}{\partial y'} = \frac{y'}{\sqrt{1 + (y')^2}}$$

Euler-Lagrange 方程：

$$0 - \frac{d}{dx} \left(\frac{y'}{\sqrt{1 + (y')^2}} \right) = 0$$

$$\Rightarrow \frac{y'}{\sqrt{1 + (y')^2}} = C \quad (\text{常数})$$

$$\Rightarrow y' = \text{常数} \Rightarrow y = ax + b \quad (\text{直线!})$$

例 1 数值验证

边界条件： $(0, 0) \rightarrow (1, 1)$

E-L 解： $y = x$ (直线)

$$L = \int_0^1 \sqrt{1 + 1} dx = \sqrt{2} \approx 1.414$$

尝试其他曲线：

$$y = x^2: y' = 2x$$

$$L = \int_0^1 \sqrt{1 + 4x^2} dx = \frac{1}{4} [2x\sqrt{1 + 4x^2} + \sinh^{-1}(2x)]_0^1 \approx 1.479$$

$$y = \sin\left(\frac{\pi x}{2}\right): y' = \frac{\pi}{2} \cos\left(\frac{\pi x}{2}\right)$$

$$L = \int_0^1 \sqrt{1 + \frac{\pi^2}{4} \cos^2\left(\frac{\pi x}{2}\right)} dx \approx 1.534$$

结论：直线确实最短！

例 2：最速降线

问题：小球从原点滑到 (x_1, y_1) ，重力场中，哪条路最快？

速度 $v = \sqrt{2gy}$ (能量守恒)，时间：

$$T[y] = \int_0^{x_1} \frac{\sqrt{1 + (y')^2}}{\sqrt{2gy}} dx$$

这里 $F = \frac{\sqrt{1 + (y')^2}}{\sqrt{2gy}}$

E-L 方程的解：摆线 (Cycloid) 参数形式

$$x = r(\theta - \sin \theta), \quad y = r(1 - \cos \theta)$$

历史：1696 年, Johann Bernoulli 提出, Newton、Leibniz、L'Hôpital、Jakob Bernoulli 都给出了解答

例 2 数值验证

边界： $(0, 0) \rightarrow (1, 1)$, $g = 10$

直线 $y = x$ ：

$$T_{\text{直线}} = \int_0^1 \frac{\sqrt{2}}{\sqrt{20x}} dx = \frac{\sqrt{2}}{\sqrt{20}} \cdot 2\sqrt{x} \Big|_0^1 = \frac{2\sqrt{2}}{\sqrt{20}} \approx \mathbf{0.632 \text{ s}}$$

摆线 (数值积分)：

$$T_{\text{摆线}} \approx \mathbf{0.584 \text{ s}}$$

摆线比直线快约 8%!

直觉： 摆线开始时更陡，小球先获得更大速度

Beltrami 恒等式：特殊情况

当 F 不显含 x 时 ($\frac{\partial F}{\partial x} = 0$)，有更简单的形式：

$$F - y' \frac{\partial F}{\partial y'} = C \quad (\text{常数})$$

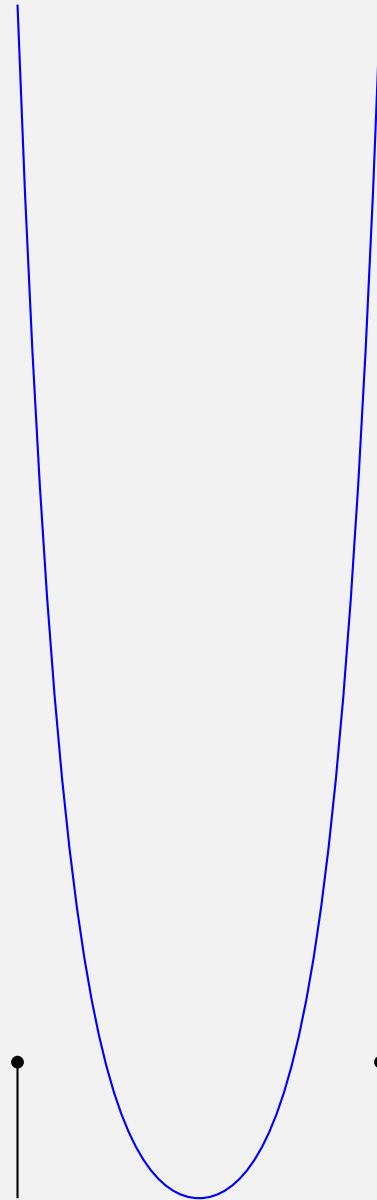
证明 (选读)：

$$\begin{aligned} \frac{d}{dx} \left(F - y' \frac{\partial F}{\partial y'} \right) &= \frac{\partial F}{\partial y} y' + \frac{\partial F}{\partial y'} y'' - y'' \frac{\partial F}{\partial y'} - y' \frac{d}{dx} \frac{\partial F}{\partial y'} \\ &= y' \left(\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} \right) = 0 \end{aligned}$$

应用： 最速降线、测地线等问题可以用这个简化

例 3：悬链线 (Catenary)

问题： 两端固定的柔软链条，在重力下的形状是什么？



悬链线

物理：链条在势能最低的位置平衡

泛函：势能 $= \int \rho g y \, ds = \int \rho g y \sqrt{1 + (y')^2} \, dx$

这里 $F = y \sqrt{1 + (y')^2}$

悬链线的求解

用 Beltrami 恒等式 (F 不显含 x):

$$F - y' \frac{\partial F}{\partial y'} = C$$

计算：

$$\frac{\partial F}{\partial y'} = \frac{y \cdot y'}{\sqrt{1 + (y')^2}}$$

$$F - y' \frac{\partial F}{\partial y'} = y \sqrt{1 + (y')^2} - \frac{y(y')^2}{\sqrt{1 + (y')^2}} = \frac{y}{\sqrt{1 + (y')^2}} = C$$

解出： $y = C \sqrt{1 + (y')^2}$

令 $y' = \sinh(t)$, 则 $y = C \cosh(t)$

悬链线方程： $y = a \cosh\left(\frac{x}{a}\right)$

悬链线数值验证

边界： $(-1, 1.5) \rightarrow (1, 1.5)$

悬链线： $y = a \cosh(x/a)$, 其中 $a \cosh(1/a) = 1.5$

数值求解： $a \approx 0.83$

$y(x) = 0.83 \cosh(x/0.83)$

势能 (取 $\rho g = 1$):

$$E_{\text{悬链线}} = \int_{-1}^1 y \sqrt{1 + (y')^2} dx \approx \mathbf{3.52}$$

对比抛物线： $y = 0.5x^2 + 1$ (同样经过两 endpoint)

$$E_{\text{抛物线}} = \int_{-1}^1 (0.5x^2 + 1) \sqrt{1 + x^2} dx \approx \mathbf{3.58}$$

悬链线势能更低! ($3.52 < 3.58$)

例 4: 等周问题 (Isoperimetric)

问题：给定周长 L , 围成最大面积的曲线是什么?

约束： $\oint ds = L$ (周长固定)

目标： $\max A = \frac{1}{2} \oint (x dy - y dx)$

这是带约束的泛函优化!

用拉格朗日乘子:

$$\mathcal{L} = A - \lambda(\quad - L)$$

结论：曲率恒定 $\Rightarrow$ 圆!

数值验证：周长 $L = 2\pi$

圆：半径 $r = 1$, 面积 $= \pi \approx \mathbf{3.14}$

正方形：边长 $= \pi/2$, 面积 $= (\pi/2)^2 \approx \mathbf{2.47}$

正六边形：面积 $\approx \mathbf{2.72}$

圆确实最大!

多变量推广

多个函数： $J[y_1, \dots, y_n] = \int F(y_1, \dots, y_n, y'_1, \dots, y'_n, x) dx$

E-L 方程（对每个 y_i ）：

$$\frac{\partial F}{\partial y_i} - \frac{d}{dx} \frac{\partial F}{\partial y'_i} = 0, \quad i = 1, \dots, n$$

高阶导数： $J[y] = \int F(y, y', y'', x) dx$

E-L 方程：

$$\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} + \frac{d^2}{dx^2} \frac{\partial F}{\partial y''} = 0$$

3 最大熵原理：约束下的分布选择

熵的回顾

离散熵： $H(p) = -\sum_i p_i \log p_i$

连续熵（微分熵）： $H[p] = -\int p(x) \log p(x) dx$

熵的含义：

- 不确定性的度量
- 信息量的期望
- 分布的“扩散程度”

熵最大的分布： 最“均匀”、最“无偏”

最大熵原理

问题： 已知某些约束（如均值、方差），应该选哪个分布？

最大熵原理（Jaynes, 1957）：

选择满足约束的**熵最大**的分布

理由：

- 不引入额外假设
- 最“保守”的选择
- 信息论上最优

数学形式：

$$\max_p H[p] = - \int p(x) \log p(x) dx$$

subject to 约束条件

例 1：无约束最大熵

问题：在 $[a, b]$ 上，哪个分布熵最大？

约束： $\int_a^b p(x) dx = 1$ (归一化)

Lagrangian：

$$\mathcal{L}[p] = - \int p \log p dx - \lambda \left(\int p dx - 1 \right)$$

变分： $\frac{\delta \mathcal{L}}{\delta p} = -\log p - 1 - \lambda = 0$

$\Rightarrow p(x) = e^{-1-\lambda} = \text{常数}$

归一化： $p(x) = \frac{1}{b-a}$ (均匀分布!)

例 1 数值验证

区间 $[0, 1]$

均匀分布： $p(x) = 1$

$H = - \int_0^1 1 \cdot \log 1 dx = 0$ (以 e 为底)

换成 bits： $H = \log_2(1) = 0$ bits... 不对!

更正：微分熵可以是负数！对于均匀分布：

$$H = - \int_0^1 1 \cdot \ln 1 dx = 0 \text{ nats}$$

对比三角分布： $p(x) = 2x$ ($x \in [0, 1]$)

$$H = - \int_0^1 2x \ln(2x) dx = - \int_0^1 2x (\ln 2 + \ln x) dx$$

$$= -\ln 2 - \int_0^1 2x \ln x dx = -\ln 2 - 2 \cdot \left(-\frac{1}{4}\right) = -\ln 2 + 0.5 \approx -0.193$$

均匀分布熵更大 ($0 > -0.193$)

例 2：给定均值的最大熵

问题： $x \geq 0$ ，已知 $\mathbb{E}[x] = \mu$ ，哪个分布熵最大？

约束：

- $\int_0^\infty p(x) dx = 1$
- $\int_0^\infty x \cdot p(x) dx = \mu$

Lagrangian：

$$\mathcal{L} = - \int p \log p dx - \lambda_0 \left(\int p dx - 1 \right) - \lambda_1 \left(\int xp dx - \mu \right)$$

变分： $-\log p - 1 - \lambda_0 - \lambda_1 x = 0$

$$\Rightarrow p(x) = e^{-1-\lambda_0} \cdot e^{-\lambda_1 x} = C e^{-\lambda_1 x}$$

这是指数分布！ $p(x) = \frac{1}{\mu} e^{-x/\mu}$

例 2 数值验证

给定 $\mu = 2$

指数分布： $p(x) = 0.5e^{-x/2}$

熵： $H = -\int_0^\infty p \ln p dx = 1 + \ln \mu = 1 + \ln 2 \approx \mathbf{1.693}$ nats

对比 Gamma 分布： $p(x) = \frac{x}{\theta^2} e^{-x/\theta}$ ，均值 $= 2\theta = 2 \Rightarrow \theta = 1$

$$p(x) = x e^{-x}$$

熵： $H = 2 - \ln(1) - \psi(2) = 2 - 0 - (1 - \gamma) \approx 2 - 0.423 = \mathbf{1.577}$ nats

指数分布熵更大 ($1.693 > 1.577$)

例 3：给定均值和方差的最大熵

问题： $x \in \mathbb{R}$ ，已知 $\mathbb{E}[x] = \mu$ ， $\text{Var}(x) = \sigma^2$

约束：

- $\int p dx = 1$
- $\int x p dx = \mu$
- $\int (x - \mu)^2 p dx = \sigma^2$

变分： $-\log p - 1 - \lambda_0 - \lambda_1 x - \lambda_2 (x - \mu)^2 = 0$

$$\Rightarrow p(x) \propto \exp(-\lambda_1 x - \lambda_2 (x - \mu)^2)$$

$$\text{配方得： } p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

这是高斯分布！

最大熵与指数族

核心结论：最大熵分布的一般形式

约束 $\mathbb{E}[f_k(x)] = \alpha_k$ ， $k = 1, \dots, K$ 下的最大熵分布是：

$$p(x) = \frac{1}{Z} \exp\left(-\sum_{k=1}^K \lambda_k f_k(x)\right)$$

这正是指数族分布的形式！

经典结果：

约束	最大熵分布	名称
无	均匀	Uniform
$\mathbb{E}[x] = \mu \ (x \geq 0)$	指数	Exponential
$\mathbb{E}[x], \text{Var}(x)$	高斯	Gaussian
$\mathbb{E}[\log x], \mathbb{E}[x]$	Gamma	Gamma
$p_i \geq 0, \sum p_i = 1$	离散均匀	Categorical

为什么高斯分布如此常见？

最大熵解释：

如果只知道均值和方差，高斯是“最保守”的选择

中心极限定理解释：

大量独立随机变量之和趋向高斯

两者的联系：

CLT 说高斯是“最可能”出现的

MaxEnt 说高斯是“最少假设”的

都指向同一个分布！

4 KL 散度与变分推断

KL 散度的定义

KL 散度 (Kullback-Leibler Divergence)：

$$D_{\text{KL}}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_p \left[\log \frac{p(x)}{q(x)} \right]$$

性质：

- $D_{\text{KL}}(p\|q) \geq 0$ (非负)
- $D_{\text{KL}}(p\|q) = 0$ 当且仅当 $p = q$
- **不对称**： $D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$

直觉： 用 q 编码来自 p 的数据，平均多花多少 bits

KL 散度的数值例子

两个高斯分布： $p = \mathcal{N}(0, 1)$, $q = \mathcal{N}(\mu, \sigma^2)$

$$D_{\text{KL}}(p||q) = \frac{1}{2} \left(\frac{1}{\sigma^2} + \frac{\mu^2}{\sigma^2} - 1 - \log \frac{1}{\sigma^2} \right)$$

数值： $p = \mathcal{N}(0, 1)$, $q = \mathcal{N}(1, 1)$

$$D_{\text{KL}}(p||q) = \frac{1}{2}(1 + 1 - 1 - 0) = \mathbf{0.5 \text{ nats}}$$

反过来： $D_{\text{KL}}(q||p) = \frac{1}{2}(1 + 1 - 1 - 0) = \mathbf{0.5 \text{ nats}}$

(这个特例下恰好相等，因为方差相同)

不同方差： $p = \mathcal{N}(0, 1)$, $q = \mathcal{N}(0, 2)$

$$D_{\text{KL}}(p||q) = \frac{1}{2}(0.5 + 0 - 1 - \ln 0.5) = \frac{1}{2}(-0.5 + 0.693) = \mathbf{0.097}$$

$$D_{\text{KL}}(q||p) = \frac{1}{2}(2 + 0 - 1 - \ln 2) = \frac{1}{2}(1 - 0.693) = \mathbf{0.153}$$

不对称!

变分推断的动机

贝叶斯推断问题：

已知观测 $\mathbf{x}$ ，求后验 $p(\mathbf{z}|\mathbf{x})$

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{\int p(\mathbf{x}|\mathbf{z})p(\mathbf{z}) d\mathbf{z}}$$

问题：分母的积分通常无法计算!

例： $\mathbf{z}$ 是 1000 维，积分是 1000 重积分

解决方案：

- MCMC：采样近似，但慢
- 变分推断：用简单分布 $q(\mathbf{z})$ 近似 $p(\mathbf{z}|\mathbf{x})$

变分推断：核心思想

目标：找 $q(\mathbf{z})$ 使得 $q(\mathbf{z}) \approx p(\mathbf{z}|\mathbf{x})$

度量近似好坏：KL 散度

$$D_{\text{KL}}(q||p) = \int q(\mathbf{z}) \log \frac{q(\mathbf{z})}{p(\mathbf{z}|\mathbf{x})} d\mathbf{z}$$

展开：

$$\begin{aligned} D_{\text{KL}}(q||p) &= \int q \log q d\mathbf{z} - \int q \log p(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\ &= \int q \log q d\mathbf{z} - \int q \log \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})} d\mathbf{z} \\ &= \int q \log q d\mathbf{z} - \int q \log p(\mathbf{x}, \mathbf{z}) d\mathbf{z} + \log p(\mathbf{x}) \end{aligned}$$

ELBO: Evidence Lower Bound

整理上式：

$$\log p(\mathbf{x}) = D_{\text{KL}}(q\|p) + \underbrace{\int q \log \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} d\mathbf{z}}_{\text{ELBO}(q)}$$

因为 $D_{\text{KL}} \geq 0$ ：

$$\log p(\mathbf{x}) \geq \text{ELBO}(q)$$

核心结论：ELBO

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})]$$

最小化 KL $\Leftrightarrow$ 最大化 ELBO**ELBO 的两种形式**

形式 1：

$$\text{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] + H[q]$$

= 联合对数似然的期望 + q 的熵

形式 2：

$$\text{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q(\mathbf{z})\|p(\mathbf{z}))$$

= 重构项 - 正则项

这正是 VAE 的损失函数！**VAE 损失函数的来源****变分自编码器 (VAE)：**编码器： $q_\phi(\mathbf{z}|\mathbf{x})$ (用神经网络参数化)解码器： $p_\theta(\mathbf{x}|\mathbf{z})$ **损失函数：**

$$\mathcal{L}(\theta, \phi) = -\text{ELBO} = -\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$$

第一项：重构损失 (让解码器能还原输入)**第二项：KL 正则** (让编码分布接近先验)**变分推断给出了 VAE 的理论基础！**

数值例子：高斯 VAE**设置：**先验 $p(\mathbf{z}) = \mathcal{N}(0, I)$ 编码器输出 $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mu_\phi(\mathbf{x}), \text{diag}(\sigma_\phi^2(\mathbf{x})))$ **KL 项**（解析形式）：对于 d 维高斯

$$D_{\text{KL}}(q||p) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$$

数值： $d = 2$, $\mu = (1, 0.5)$, $\sigma^2 = (0.5, 0.8)$

$$D_{\text{KL}} = \frac{1}{2}[(1 + 0.5 - 1 - \ln 0.5) + (0.25 + 0.8 - 1 - \ln 0.8)]$$

$$= \frac{1}{2}[(0.5 + 0.693) + (0.05 + 0.223)] = \frac{1}{2}[1.193 + 0.273] = \mathbf{0.733}$$

5 信息瓶颈

信息瓶颈的动机**问题：** 给定输入 X 和目标 Y ，如何找最好的表示 Z ？**好的表示应该：**

- 保留关于 Y 的信息（预测能力）
- 压缩 X 的冗余信息（简洁性）

信息论度量：

- $I(Z; Y)$: Z 关于 Y 的信息（越大越好）
- $I(X; Z)$: Z 关于 X 的信息（越小越好，压缩）

信息瓶颈目标**Tishby et al., 1999:**

$$\min_{p(z|x)} I(X; Z) - \beta I(Z; Y)$$

 $\beta > 0$ **控制权衡：**

- β 大：更看重预测能力
- β 小：更看重压缩

等价形式：

$$\max_{p(z|x)} I(Z; Y) \quad \text{s.t.} \quad I(X; Z) \leq R$$

在压缩预算 R 下最大化预测信息

信息瓶颈与深度学习

深度网络： $X \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_L \rightarrow Y$

每一层 h_l 可以看作 X 的表示

Shwartz-Ziv & Tishby, 2017:

训练过程中：

- 初期： $I(X; h_l)$ 增加（学习特征）
- 后期： $I(X; h_l)$ 减少， $I(h_l; Y)$ 保持（压缩无关信息）

这叫“信息平面”分析

（虽然后来有争议，但思想很有影响力）

信息瓶颈的数值例子

简单设置：

$X \in \{0, 1, 2, 3\}$ （4 个值）， $Y \in \{0, 1\}$ （2 个值）

$$p(x, y) = \begin{cases} 0.2 & (x, y) \in \{(0, 0), (1, 0), (2, 1), (3, 1)\} \\ 0.1 & (x, y) \in \{(0, 1), (1, 1), (2, 0), (3, 0)\} \end{cases}$$

完美表示： $Z = X$

$I(X; Z) = H(X) \approx 2$ bits（完全保留）

$I(Z; Y) = I(X; Y) \approx 0.28$ bits

压缩表示： $Z = \mathbf{1}[X \geq 2]$

$I(X; Z) = H(Z) = 1$ bit（压缩一半）

$I(Z; Y) = I(X; Y)$ （如果 Z 与 Y 关系不变）

变分信息瓶颈（VIB）

问题： $I(X; Z)$ 难以计算

VIB (Alemi et al., 2017)：用变分方法近似

$$\mathcal{L}_{\text{VIB}} = \mathbb{E}_{p(x, y)} \left[\mathbb{E}_{q_\phi(z|x)} [-\log p_\theta(y|z)] + \beta D_{\text{KL}}(q_\phi(z|x) \| r(z)) \right]$$

比较 VAE：

- VAE: 重构 x , 正则化到先验
- VIB: 预测 y , 正则化到先验

VIB 用于分类: 更鲁棒、更好的泛化

6 最优控制与 HJB 方程

最优控制问题

问题设置:

状态: $\mathbf{x}(t) \in \mathbb{R}^n$

控制: $\mathbf{u}(t) \in \mathbb{R}^m$

动力学: $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t)$

目标: 最小化代价

$$J = \phi(\mathbf{x}(T)) + \int_0^T L(\mathbf{x}, \mathbf{u}, t) dt$$

ϕ : 终端代价, L : 运行代价

找最优控制 $\mathbf{u}^*(t)$ 使 J 最小

具体例子: LQR 问题

线性动力学: $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$

二次代价:

$$J = \frac{1}{2} \mathbf{x}(T)^T Q_f \mathbf{x}(T) + \frac{1}{2} \int_0^T (\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}) dt$$

数值例子: 一维情况

$\dot{x} = x + u$ (不稳定系统)

$$J = \frac{1}{2} x(T)^2 + \frac{1}{2} \int_0^T (x^2 + u^2) dt$$

目标: 控制 x 到 0, 同时不用太大的 u

Bellman 的最优性原理

关键洞察 (Bellman, 1957):

“最优策略的子策略也是最优的”

数学表述: 定义值函数

$$V(\mathbf{x}, t) = \min_{\mathbf{u}(\cdot)} \left[\phi(\mathbf{x}(T)) + \int_t^T L(\mathbf{x}, \mathbf{u}, s) ds \right]$$

从状态 $(\mathbf{x}, t)$ 出发的最小代价

最优性原理：

$$V(\mathbf{x}, t) = \min_{\mathbf{u}} [L(\mathbf{x}, \mathbf{u}, t) dt + V(\mathbf{x} + f(\mathbf{x}, \mathbf{u}, t) dt, t + dt)]$$

HJB 方程的推导

Taylor 展开：

$$V(\mathbf{x} + f dt, t + dt) \approx V + \frac{\partial V}{\partial t} dt + \nabla_x V \cdot f dt$$

代入最优性原理：

$$V = \min_{\mathbf{u}} \left[L dt + V + \frac{\partial V}{\partial t} dt + \nabla_x V \cdot f dt \right]$$

消去 V ，除以 dt ：

$$0 = \min_{\mathbf{u}} \left[L + \frac{\partial V}{\partial t} + \nabla_x V \cdot f \right]$$

Hamilton-Jacobi-Bellman 方程

核心结论：HJB 方程

$$-\frac{\partial V}{\partial t} = \min_{\mathbf{u}} [L(\mathbf{x}, \mathbf{u}, t) + \nabla_x V \cdot f(\mathbf{x}, \mathbf{u}, t)]$$

边界条件： $V(\mathbf{x}, T) = \phi(\mathbf{x})$

这是一个偏微分方程 (PDE)

从终端 $t = T$ 向初始 $t = 0$ 求解 (反向!)

难点：维度灾难—— V 定义在状态空间上，高维时难以求解

LQR 的 HJB 解

猜测： $V(\mathbf{x}, t) = \frac{1}{2} \mathbf{x}^T P(t) \mathbf{x}$

代入 HJB：

$$\nabla_x V = P \mathbf{x}$$

$$\frac{\partial V}{\partial t} = \frac{1}{2} \mathbf{x}^T \dot{P} \mathbf{x}$$

最小化对 $\mathbf{u}$ ：

$$\mathbf{u}^* = -R^{-1} B^T P \mathbf{x} \quad (\text{线性反馈!})$$

代入得 Riccati 方程：

$$-\dot{P} = Q + A^T P + P A - P B R^{-1} B^T P$$

边界条件： $P(T) = Q_f$

LQR 数值例子

一维： $\dot{x} = x + u$, $Q = R = Q_f = 1$

Riccati 方程: $-\dot{P} = 1 + 2P - P^2$

稳态 ($\dot{P} = 0$): $P^2 - 2P - 1 = 0$

$P = 1 + \sqrt{2} \approx 2.414$

最优控制: $u^* = -Px = -2.414x$

闭环动力学: $\dot{x} = x - 2.414x = -1.414x$

稳定! (极点在 -1.414)

不加控制: $\dot{x} = x$, 指数增长, 不稳定

7 HJB $\rightarrow$ Bellman, 伴随 $\rightarrow$ 反向传播**离散时间: Bellman 方程**

离散化 HJB:

时间 $t_k = k \cdot \Delta t$, $k = 0, 1, \dots, N$

值函数: $V_k(\mathbf{x}) =$ 从 $(\mathbf{x}, t_k)$ 出发的最优代价

Bellman 方程:

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} [L(\mathbf{x}, \mathbf{u})\Delta t + V_{k+1}(f(\mathbf{x}, \mathbf{u}))]$$

边界: $V_N(\mathbf{x}) = \phi(\mathbf{x})$

这正是强化学习中的 **Bellman 方程!**

RL 中的 Bellman 方程

RL 术语:

状态 s , 动作 a , 奖励 r , 转移 $P(s'|s, a)$

值函数: $V(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s]$

Bellman 方程:

$$V(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

对比最优控制:

最优控制	强化学习
状态 $\mathbf{x}$	状态 s
控制 $\mathbf{u}$	动作 a
代价 L	负奖励 $-r$
动力学 f	转移 P
min 代价	max 奖励

核心联系：HJB = 连续 Bellman

HJB 方程（连续时间）：

$$-\frac{\partial V}{\partial t} = \min_{\mathbf{u}} [L + \nabla V \cdot f]$$

Bellman 方程（离散时间）：

$$V_k = \min_{\mathbf{u}} [L\Delta t + V_{k+1}(f)]$$

关系：

$\Delta t \rightarrow 0$ 时, Bellman $\rightarrow$ HJB

HJB 是 Bellman 的**连续时间极限**！

这为 RL 提供了理论基础

伴随方法 (Adjoint Method)

问题： 计算 $\frac{dJ}{d\theta}$ ，其中

$$J = \phi(\mathbf{x}(T)), \quad \dot{\mathbf{x}} = f(\mathbf{x}, \theta, t)$$

θ 是参数（如神经网络权重）

直接方法：

对每个 θ_i ，扰动并重新积分 $\mathbf{x}(t)$

复杂度： $O(p \cdot T)$ ， p 是参数数量

伴随方法：

引入伴随状态 $\mathbf{a}(t) = \frac{\partial J}{\partial \mathbf{x}(t)}$

复杂度： $O(T)$ ，与参数数量无关！

伴随方程

伴随状态： $\mathbf{a}(t) = \frac{\partial J}{\partial \mathbf{x}(t)}$

伴随方程（反向 ODE）：

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^T \frac{\partial f}{\partial \mathbf{x}}$$

终端条件： $\mathbf{a}(T) = \frac{\partial \phi}{\partial \mathbf{x}(T)}$

参数梯度：

$$\frac{dJ}{d\theta} = \int_0^T \mathbf{a}(t)^T \frac{\partial f}{\partial \theta} dt$$

计算流程：

1. 前向：积分 $\dot{\mathbf{x}} = f$ 从 0 到 T
2. 反向：积分伴随方程从 T 到 0
3. 累积梯度

伴随方法 = 反向传播

神经网络： $\mathbf{h}_{l+1} = f_l(\mathbf{h}_l, \theta_l)$

损失： $J = \mathcal{L}(\mathbf{h}_L)$

反向传播：

$$\frac{\partial J}{\partial \mathbf{h}_l} = \frac{\partial J}{\partial \mathbf{h}_{l+1}} \cdot \frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l}$$

对比伴随方程：

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^T \frac{\partial f}{\partial \mathbf{x}}$$

离散化：

$$\mathbf{a}_l = \mathbf{a}_{l+1}^T \frac{\partial f_l}{\partial \mathbf{h}_l}$$

完全一样！ 反向传播是伴随方法的**离散形式**

Neural ODE

Chen et al., 2018：把神经网络看作 ODE

传统 ResNet： $\mathbf{h}_{l+1} = \mathbf{h}_l + f_l(\mathbf{h}_l)$

连续极限： $\frac{d\mathbf{h}}{dt} = f(\mathbf{h}, t, \theta)$

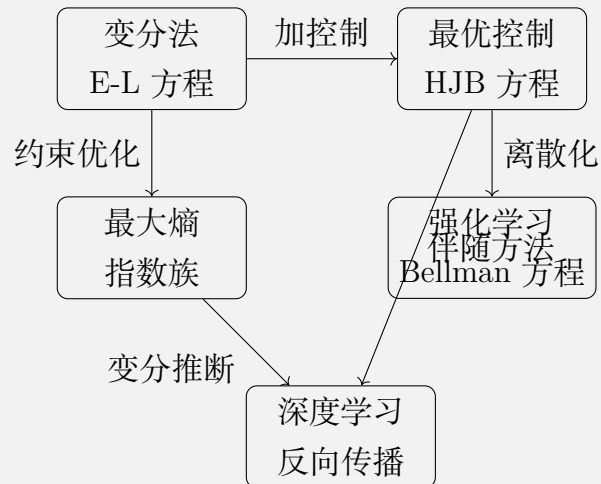
优势：

- 内存 $O(1)$ （用伴随方法，不存中间状态）
- 连续深度（可以选任意精度）
- 可逆（用于生成模型）

代价：

- 训练慢（需要积分 ODE）
- 数值稳定性问题

统一视角



一切都是优化！

8 实验

实验：实验 10a: Neural ODE 基础 (12 分钟)

目标：用 torchdiffeq 实现 Neural ODE

任务 1：拟合螺旋线数据

```

1 import torch
2 from torchdiffeq import odeint
3
4 # 定义 ODE 函数
5 class ODEFunc(torch.nn.Module):
6     def __init__(self):
7         super().__init__()
8         self.net = torch.nn.Sequential(
9             torch.nn.Linear(2, 50),
10            torch.nn.Tanh(),
11            torch.nn.Linear(50, 2)
12        )
13
14     def forward(self, t, y):

```

```
15         return self.net(y)
16
17 # 生成螺旋线数据
18 def generate_spiral(n_points=100):
19     t = torch.linspace(0, 4*torch.pi, n_points)
20     x = t * torch.cos(t)
21     y = t * torch.sin(t)
22     return torch.stack([x, y], dim=1)
23
24 # 训练
25 func = ODEFunc()
26 optimizer = torch.optim.Adam(func.parameters(), lr=0.01)
27 true_y = generate_spiral()
28 y0 = true_y[0:1]
29 t = torch.linspace(0, 1, 100)
30
31 for epoch in range(1000):
32     pred_y = odeint(func, y0, t)
33     loss = ((pred_y.squeeze() - true_y)**2).mean()
34     optimizer.zero_grad()
35     loss.backward()
36     optimizer.step()
```

实验：实验 10a 续：分析结果

任务 2：比较不同求解器

```
1 # 不同求解器
2 methods = ['euler', 'midpoint', 'rk4', 'dopri5']
3
4 for method in methods:
5     pred = odeint(func, y0, t, method=method)
6     # 计算误差和时间
```

预期结果：

求解器	相对误差	时间
euler	10^{-2}	1x
midpoint	10^{-3}	2x
rk4	10^{-5}	4x
dopri5	10^{-7}	可变

思考：精度 vs 速度的权衡

实验：实验 10b: Adjoint 方法对比 (12 分钟)

目标：比较直接反传 vs Adjoint 方法的显存

任务：随深度增加，观察显存变化

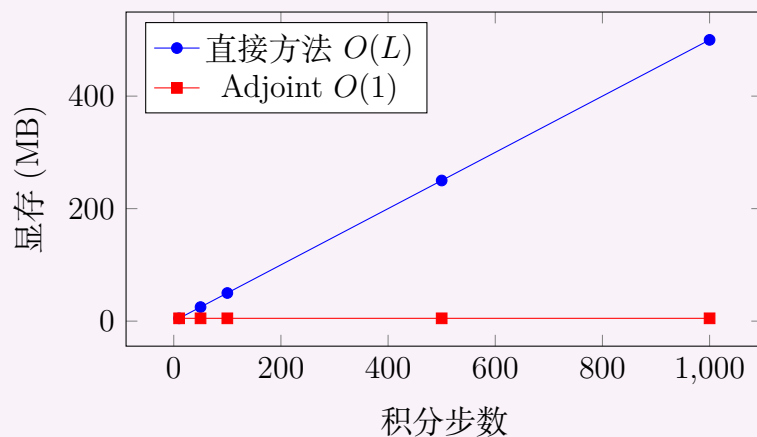
```

1 from torchdiffeq import odeint, odeint_adjoint
2
3 # 不同深度 (积分步数)
4 depths = [10, 50, 100, 500, 1000]
5
6 for depth in depths:
7     t = torch.linspace(0, 1, depth)
8
9     # 直接方法
10    torch.cuda.reset_peak_memory_stats()
11    pred = odeint(func, y0, t)
12    loss = pred[-1].sum()
13    loss.backward()
14    mem_direct = torch.cuda.max_memory_allocated()
15
16    # Adjoint 方法
17    torch.cuda.reset_peak_memory_stats()
18    pred = odeint_adjoint(func, y0, t)
19    loss = pred[-1].sum()
20    loss.backward()
21    mem_adjoint = torch.cuda.max_memory_allocated()
22
23    print(f"Depth {depth}: Direct={mem_direct/1e6:.1f}MB, Adjoint={mem_adjoint/1e6:.1f}MB")

```

实验：实验 10b 续：显存分析

预期结果：



关键观察：

直接方法：显存 $\propto$ 深度（需要存所有中间状态）

Adjoint：显存恒定（只需当前状态）

代价：Adjoint 需要重新计算前向，时间约 2 倍

实验：实验 10c：连续 Normalizing Flow（6 分钟）

目标：用 Neural ODE 做密度估计

核心公式：对于 ODE $\frac{dz}{dt} = f(z, t)$

$$\frac{d \log p(\mathbf{z})}{dt} = -\text{tr} \left(\frac{\partial f}{\partial \mathbf{z}} \right)$$

任务：拟合二维分布

```

1 class CNF(torch.nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.net = torch.nn.Sequential(...)
5
6     def forward(self, t, states):
7         z, log_p = states[0], states[1]
8         dz_dt = self.net(torch.cat([z, t.expand(z.shape[0], 1)],
9                                     dim=1))
10
11         # 计算迹（用Hutchinson估计）
12         e = torch.randn_like(z)
13         e_dz = torch.autograd.grad(dz_dt, z, e, create_graph=

```

```

        True)[0]
        trace = (e_dz * e).sum(dim=1)

    return dz_dt, -trace

# 从简单分布（高斯）变换到复杂分布
z0 = torch.randn(1000, 2)
log_p0 = -0.5 * (z0**2).sum(dim=1)
z1, log_p1 = odeint(cnf, (z0, log_p0), torch.tensor([0., 1.]))

```

9 本周总结

核心结论：核心公式

1. Euler-Lagrange 方程：

$$\frac{\partial F}{\partial y} - \frac{d}{dx} \frac{\partial F}{\partial y'} = 0$$

2. 最大熵分布：

$$p(x) \propto \exp \left(- \sum_k \lambda_k f_k(x) \right)$$

3. ELBO：

$$\text{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q\|p)$$

4. HJB 方程：

$$-\frac{\partial V}{\partial t} = \min_{\mathbf{u}} [L + \nabla V \cdot f]$$

5. 伴随方程：

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^T \frac{\partial f}{\partial \mathbf{x}}$$

核心结论：核心联系

概念	联系
E-L 方程	泛函优化的必要条件
最大熵	→ 指数族分布
变分推断	$\min \text{KL} \Leftrightarrow \max \text{ELBO}$
HJB 方程	→ Bellman 方程 (RL)
伴随方法	= 反向传播

统一视角：优化 + 动力系统 + 概率

10 思考题

课后思考

概念题：

1. 为什么高斯分布在给定均值和方差下熵最大？直觉解释？
2. KL 散度为什么不对称？ $D_{\text{KL}}(p\|q)$ 和 $D_{\text{KL}}(q\|p)$ 分别在什么场景下使用？
3. 伴随方法为什么能节省显存？代价是什么？

计算题：

4. 求解最短路径问题：从 $(0, 0)$ 到 $(1, 2)$ 的最短曲线
5. 给定 $\mathbb{E}[x] = 1$, $\mathbb{E}[x^2] = 2$, 求最大熵分布
6. 对于 LQR: $\dot{x} = 2x + u$, $J = \int_0^\infty (x^2 + u^2)dt$, 求最优控制

编程题：

7. 实现一个简单的 Neural ODE 分类器，与普通 ResNet 比较
8. 用变分信息瓶颈训练分类器，比较鲁棒性